# index

A

AbstractMessageRouter bean 256

access() method 126

action attribute 41

activating profiles 158–159

*ActiveMQ in Action* (Snyder, Bosanac, and Davies) 211

Actuator 388–391

configuring base path 389–390

consuming endpoints 391–408

application information 392–395

runtime metrics 405–408

viewing application activity 403–405

viewing configuration details 395–403

customizing 408–419

endpoints 417–419

health indicators 414–415

registering metrics 415–417

the /info endpoint 408–413

enabling and disabling endpoints 390–391

securing 420–421

Actuator MBeans

creating 437–440

working with 435–437

addIngredient() method 207

addIngredientsToModel() method 70

addNote() method 419

addScript() method 141

addScripts() method 141

addTaco() method 34, 348

addViewController() method 56

addViewControllers() method 56, 129

administering Spring

Admin server 427–431

environment properties 429–431

general application health and information 428

key metrics 428–429

logging levels 431

securing 431–433

using Spring Boot Admin 424–427

creating Admin server 424–426

registering Admin clients 426–427

Admin server 427–431

creating 424–426

environment properties 429–431

general application health and information 428

key metrics 428–429

logging levels 431

securing 431–433

authenticating with Actuator 433

enabling login 432–433

aggregate root service, OrderRepository 347–353

Alert object 378

allIngredients() method 401

all operation 306

AMQP 211, 226–236

adding RabbitMQ to Spring 227–228

receiving message from RabbitMQ 232–236

handling RabbitMQ messages with listeners 235–236

receiving messages with RabbitTemplate 232–235

sending messages with RabbitTemplate 228–232

configuring message converter 231

setting message properties 231–232

and() method 129

any() operation 306

ApiProperties object 273

APIs, reactive

consuming REST APIs reactively 325–332

deleting resources 329

exchanging requests 331–332

GETting resources 326–328

handling errors 329–331

sending resources 328–329

functional request handlers 316–320

securing reactive web APIs 333–336

configuring reactive user details service 335–336

configuring reactive web security 333–335

Spring WebFlux 309–316

overview 310–311

writing reactive controllers 312–316

testing reactive controllers 320–325

GET requests 320–323

POST requests 323–324

with live server 324–325

ApplicationArguments parameter 84

applicationConfig property 398

ApplicationRunner bean 374

applications

configuration details 395–403

autoconfiguration 396–397

bean wiring report 395–396

HTTP request mappings 400–401

inspecting environment and configuration properties 397–400

managing logging levels 401–403

information 392–395

asking for 392

inspecting health 393–395

Spring

building and running projects 477

initializing project from command line 472–477

IntelliJ IDEA 463–464

NetBeans 465–469

Spring Tool Suite 459–462

start.spring.io 469–472

viewing activity 403–405

monitoring threads 404–405

tracing HTTP activity 403–404

args parameter 85

array type 341

<artifactId> property 447

assertOrder() method 352

asynchronous messaging

with JMS 211–226

JmsTemplate 214–222

receiving messages 222 – 226

setting up 211–214

with Kafka 236–242

sending messages with KafkaTemplate 238–240

setting up Spring for 237–238

writing Kafka listeners 241–242

with RabbitMQ and AMQP 226–236

adding RabbitMQ to Spring 227–228

receiving message from RabbitMQ 232–236

sending messages with RabbitTemplate 228–232

authentication 116–124

customizing user authentication 119–124

in-memory user details service 118–119

authorization code grant 190

authorization servers 192–201

authorizationServerSecurityFilterChain() bean method 195

autoconfiguration 141–148

data source 143–144

embedded server 145

logging 146–147

overview 396–397

Spring environment abstraction 142–143

using special property values 148

automatic features

automatic application restart 24

automatic browser refresh and template cache disable 24

B

base path, Actuator 389–390

base URI 327

@Bean() method 125, 141, 252, 317, 454

@Bean-annotated() method 84, 141

beans

conditionally creating with profiles 159–160

wiring report 395–396

Between operation 91

Bosanac, Dejan 211

browser refresh, automatic 24

BSON (Binary JSON) format 106

buffer() operation 302

buffering data 302–305

build() method 335

Builder object 409

build/libs directory 445

C

caching templates 59

Carnell, John 426

cascade attribute 89

Cassandra

reactively persisting data in 361–368

creating reactive Cassandra repositories 365–366

domain classes for 362–365

testing reactive Cassandra repositories 366–368

repositories 95–105

data modeling 98–99

enabling Spring Data Cassandra 95–98

mapping domain types for Cassandra persistence 99–105

writing 105

Cassandra CQL (Cassandra Query Language) 96

ccCVV property 51

ccExpiration property 51

ccNumber property 51

cf command-line tool 446

channel adapters 263–265

class attribute 54

clients

creating RSocket 372–382

fire-and-forget messages 378–379

request-response 375

request-stream messaging 376–377

developing 204–209

sending messages bidirectionally 379–382

*Cloud Native Spring in Action* (Vitale) 28

cloud profile 158

collection attribute 108

collections, creation operations from 288–290

collectList() operation 304

collectMap() operation 304

combination operations 291–295

merging 291–294

selecting first reactive type to publish 294–295

command line 472–477

curl and Initializr API 473–475

Spring Boot command-line interface 475–477

CommandLineRunner 83–85

CommandLineRunner bean 84, 159, 167, 194

@ConditionalOnMissingBean annotation 397

configuration() method 333

@Configuration-annotated class 160

@Configuration annotation 5

@Configuration class 317

configuration properties

autoconfiguration 141–148

data source 143–144

embedded server 145

logging 146–147

Spring environment abstraction 142–143

using special property values 148

creating 148–155

declaring metadata 153–155

defining holders 151–153

inspecting 397–400

with profiles 155–160

activating 158–159

conditionally creating beans with 159–160

profile-specific properties 156–157

@ConfigurationProperties-annotated class 234

@ConfigurationProperties annotation 148

ConfigurationPropertiesAutoConfiguration bean 397

configure() method 334, 420, 456

configuredLevel property 402

consumes attribute 170

container images 446–455

deploying to Kubernetes 449–451

enabling graceful shutdown 451–452

liveness and readiness 452–455

configuring probes in deployment 453–454

enabling probes 452–453

managing 454–455

contexts element 396

contribute() method 409

controllers

creating class for 34–38

testing 20–21

view controllers 54–57

writing reactive 312–316

handling input reactively 315–316

returning single values 314

RxJava types 314–315

writing RESTful controllers 164–174

deleting data from server 173–174

retrieving data from server 164–170

sending data to server 170–171

updating data on server 171–173

convert() method 43

convertAndSend() method 215

Converter interface 43

converters, message

configuring with RabbitTemplate 231

JmsTemplate

before sending 218–219

configuring 219–220

core Spring Framework 26

count() method 409

createdAt property 101

createdDate property 179

create keyspace command 96, 361

creation operations 287–291

from collections 288–290

from objects 287–288

generating Flux data 290–291

@CrossOrigin annotation 166

cross-site request forgery 133–134

CrudRepository interface 89, 343

curl command 402, 473

curl command-line client 391

curl option 477

D

data

buffering on reactive stream 302–305

filtering from reactive types 295–299

mapping reactive 299–302

nonrelational data

Cassandra repositories 95–105

MongoDB repositories 106–111

persisting data with Spring Data JPA 85–93

adding to project 85–86

annotating domain as entities 86–89

customizing repositories 90–93

declaring JPA repositories 89–90

persisting document data reactively 353–360

domain document types 354–356

reactive MongoDB repositories 356–357

testing reactive MongoDB repositories 357–360

reading and writing data with JDBC 62–78

adapting domain for persistence 64–65

inserting data 73–78

JdbcTemplate 65–70

schema and preloading data 70–73

RESTful controllers

deleting data from server 173–174

retrieving data from server 164–170

sending data to server 170–171

updating data on server 171–173

Spring Data JDBC 78–85

adding to build 78–79

annotating domain for persistence 81–83

preloading data with CommandLineRunner 83–85

repository interfaces 79–80

@Data annotation 32, 87

data-backed services 174–180

adjusting resource paths and relation names 177

paging and sorting 180

@DataMongoTest annotation 359

data parameter 245

@DataR2dbcTest annotation 345

DataSource bean 141

Davies, Rob 211

DDD (domain-driven design) 71

defaultRequestChannel attribute 245

delayElements() operation 292

delaySubscription() operation 292

delete() method 184, 329

deleteAllOrders() method 135

deleteById() method 174

deleteNote() method 419

deleteOrder() method 174

DELETE request 173, 201, 400

deleting resources 184, 329

deliveryName property 82

deliveryZip property 90

dependencies 286

<dependencies> element 14

--dependencies parameter 476

dependencies parameter 475

dependencies property 396

deploying Spring

container images 446–455

deploying to Kubernetes 449–451

enabling graceful shutdown 451–452

liveness and readiness 452 - 455

executable JAR files 445–446

options for 444–445

WAR files 455–457

description property 154

DesignTacoController class 35

Destination bean 217

Destination object 217

dev profile 159

DI (dependency injection) 4

diagramming reactive flows 285–286

disabling endpoints 390–391

display, web applications 30–41

creating controller class 34–38

designing view 38–41

establishing domain 31–34

distinct() operation 298

 element 39

docker build command 446

@Document annotation 108, 354

document data 353–360

domain document types 354–356

reactive MongoDB repositories 356–357

testing reactive MongoDB repositories 357–360

domain-driven design (DDD) 71

Domain-Driven Design (Evans) 31, 71

domains

adapting for persistence 64–65

annotating as entities 86–89

annotating for persistence 81–83

entities for R2DBC 339–343

establishing 31–34

for Cassandra persistence 99–105, 362–365

mapping MongoDB repositories to documents 107–110

doTransform() method 272

DSL configuration 249–251

E

effectiveLevel property 402

email integration flow 267–274

EmailOrder class 272

EmailOrder object 272

EmailProperties class 268

EmailToOrderTransformer class 270

enabling endpoints 390–391

EndpointRequest.toAnyEndpoint() method 421

endpoints 391–408

application activity 403–405

monitoring threads 404–405

tracing HTTP activity 403–404

application information 392–395

asking for 392

inspecting application health 393–395

configuration details 395–403

autoconfiguration 396–397

bean wiring report 395–396

HTTP request mappings 400–401

inspecting environment and configuration properties 397–400

managing logging levels 401–403

customizing 417–419

enabling and disabling 390–391

modules 265 – 267

runtime metrics 405–408

the /info endpoint 408–413

build information 410–411

Git commit information 411–413

InfoContributor 408–410

*Enterprise Integration Patterns* (Hohpe and Woolf) 244

environment properties 397–400

Equals operation 91

errors 54, 329–331

Evans, Eric 31, 71

event looping 309

exception tag 408

exchangeToFlux() method 332

exchangeToMono() method 331

exchanging requests 331–332

expectBodyList() method 322

*Expert One-on-One J2EE Design and Development* (Johnson) 3

F

failOnNoGitDirectory property 411

fields property 54

FIFO (first in, first out) 252

filename parameter 245

Files type 250

FileWriterGateway interface 251

FileWritingMessageHandler bean 249

filter() method 254

filter() operation 298

@Filter annotation 254

filterChain() method 125

filtering data 295–299

filters 253–254

final property 32

findAll() method 70, 166, 207, 312

findById() method 68, 169, 343

findBySlug() method 345

findByUsername() method 121

findByUserOrderByPlacedAtDesc() method 149

fire-and-forget messages 378–379

firstWithSignal() operation 294

flatMap() operation 299, 320

Flux data 290–291

force attribute 87

forgery, cross-site request 133–134

<form> element 41, 134

forms

processing submissions 41–49

validating form input 49–54

declaring validation rules 50–52

displaying validation errors 54

performing validation at form binding 52–54

frameworkless framework 25

from() method 264

fromArray() method 289, 382

fromIterable() method 289

*Functional Programming in Java* (Saumont) 281

functional request handlers 316–320

G

Gamov, Viktor 237

gateways 262–263

GeneratedKeyHolder type 74

GenericHandler interface 273

GenericSelector interface 254

GenericTransformer interface 255

GET() method 317

get() method 169

getAuthorities() method 121

getContent() method 312

getForEntity() method 183

getForObject() method 183, 326

getImapUrl() method 270

@GetMapping-annotated() method 173

@GetMapping annotation 42, 166

getMessageConverter() method 229

GET requests 18, 35, 37–38, 123, 164, 319–323, 389, 429, 454

getTacoCount() method 439

GETting resources 182–183, 326–328

making requests with base URI 327

timing out on long-running requests 328

Git commit information 411–413

graceful shutdown, enabling 451–452

GratuityIn class 380

GratuityOut class 380

gratuity property 380

*Grokking Functional Programming* (Plachta) 281

H

H2 Console 25

handle() method 241, 273

handleGreeting() method 373

.hasAuthority() method 202

hasErrors() method 53

hasRole() method 126

@Header annotation 245

HealthIndicator interface 414

health indicators 414–415

helloRouterFunction() method 318

Hohpe, Gregor 244

holders, configuration property 151–153

home() method 18

homepages 20

HTTP

request mappings 400–401

tracing activity 403–404

HttpSecurity object 125, 334

HttpStatus object 331

I

@Id annotation 82, 108

id field 64

id parameter 169

@Id property 109

id property 76, 100, 184

if statement 310

if/then blocks 49

--imageName parameter 448

<img> tag 19

@ImportResource annotation 247

import statement 25

@InboundChannelAdapter annotation 264

increment() method 439

info.contact.email property 392

info.contact.phone property 392

InfoContributor 408–410

/info endpoint 408–413

build information 410–411

Git commit information 411–413

InfoContributor 408 - 410

Ingredient class 31, 64, 99, 339

Ingredient data 66

Ingredient entity 177

ingredientId parameter 182

Ingredient object 64, 182, 326, 340

IngredientRepository.findById() method 70

IngredientRepository interface 66, 111, 177, 356

ingredientsController bean 396

IngredientUDT class 102, 363

initializing applications 6–17

examining project structure 11–17

bootstrapping application 15–16

build specification 12–15

testing application 16–17

with Spring Tool Suite 11

in-memory user details service 118–119

<input> element 39

inputChannel attribute 253

insert() method 111

insert query 75

integration flow 244–251

configuring in Java 247–249

using DSL configuration 249–251

with XML 246–247

IntelliJ IDEA 463–464

interfaces, MongoDB repositories 111

interval() method 291

int-file namespace 247

InventoryService bean 5

is* method 121

J

JAR files, building executable 445–446

Java, configuring integration flow in 247–249

java -jar command 477

Java Message Service. *See* JMS

--java parameter 476

javax.persistence package 87

JDBC (Java Database Connectivity) 61

reading and writing data with 62–78

adapting domain for persistence 64–65

inserting data 73–78

JdbcTemplate 65–70

schema and preloading data 70–73

Spring Data JDBC 78–85

adding to build 78–79

annotating domain for persistence 81–83

preloading data with CommandLineRunner 83–85

repository interfaces 79–80

JdbcIngredientRepository bean 67

JdbcTemplate 65–70

defining JDBC repositories 66–68

inserting rows 68–70

JMS (Java Message Service) 211–226

JmsTemplate 214–222

configuring message converter 219–220

converting messages before sending 218–219

postprocessing messages 220–222

receiving messages 222–226

declaring message listeners 224–226

with JmsTemplate 222–224

setting up 211–214

@JmsListener annotation 225

JmsTemplate() method 228

JmsTemplatesConvertAndSend() method 218

JMX

Actuator MBeans

creating 437–440

working with 435–437

sending notifications 440–442

JNDI (Java Naming and Directory Interface) 144

Johnson, Rod 3

json() method 322

just() method 287

JWK (JSON Web Key) 196

JWT (JSON Web Token) 191

K

Kafka 236–242

sending messages with KafkaTemplate 238–240

setting up Spring for 237–238

writing Kafka listeners 241–242

*Kafka in Action* (Scott, Gamov, and Klein) 237

@KafkaListener annotation 241

Klein, Dave 237

kubectl command-line tool 450

kubectl port-forward command 451

Kubernetes, deploying to 449–451

*Kubernetes in Action, 2nd Edition* (Lukša) 449

L

LineItem object 259

lineItemSplitter() method 259

ListBodySpec object 323

listeners

declaring for JMS 224–226

handling RabbitMQ messages with 235–236

writing Kafka 241–242

--list parameter 476

List<Taco> property 348

liveness and readiness 452–455

configuring probes in deployment 453–454

enabling probes 452–453

managing 454–455

live servers 324–325

loadUserByUsername() method 118

log() operation 303

Logger object 45

loggers element 402

logging 401–403

logging.file.name property 147

logging.file.path property 147

logging.level.tacos property 156

logic operations 305–307

login pages 128 – 130

long parameter 233

long-running requests 328

Lukša, Marko 449

M

main() method 16

management.endpoint.health.show-details property 393

management.endpoints.web.exposure.include property 390

management.endpoint.web.base-path property 389

management.info.git.mode property 412

@ManyToOne annotation 137

map() function 373

map() method 349

map() operation 284, 336, 350

mapping

HTTP request 400–401

reactive data 299–302

mapRowToIngredient() method 68

matches() method 117

memory, in-memory user details service 118–119

mergeWith() method 292

mergeWith() operation 291

merging reactive types 291–294

message attribute 52

message channels 252–253

MessageHandler bean 261

MessageHandler interface 262

@MessageMapping annotation 373

Message object 272

MessageProperties object 232

messaging

fire-and-forget messages 378–379

request-stream messaging 376–377

sending bidirectionally 379–382

with JMS 211–226

JmsTemplate 214–222

receiving messages 222–226

setting up 211 – 214

with Kafka 236–242

sending messages with KafkaTemplate 238 – 240

setting up Spring for 237 – 238

writing Kafka listeners 241–242

with RabbitMQ and AMQP 226–236

adding RabbitMQ to Spring 227–228

receiving message from RabbitMQ 232–236

sending messages with RabbitTemplate 228–232

metadata, configuration property 153–155

method-level security 134–136

method tag 408

metrics

registering 415–417

runtime 405–408

microservices 28

MockMvc object 21

Model object 37

MongoDB

persisting document data reactively with 353–360

domain document types 354–356

reactive MongoDB repositories 356–357

testing reactive MongoDB repositories 357–360

repositories 106–111

enabling Spring Data MongoDB 106 – 107

mapping domain types to documents 107–110

writing interfaces 111

MongoTemplate bean 397

monitoring Spring

Actuator MBeans

creating 437–440

working with 435–437

sending notifications 440–442

Mono.just() method 349

Mono objects 284

Mono<String> parameter 373

Mono type 370

N

name property 39

NetBeans 465–469

nonrelational data

Cassandra repositories 95–105

Cassandra data modeling 98–99

enabling Spring Data Cassandra 95 – 98

mapping domain types for Cassandra persistence 99–105

writing 105

MongoDB repositories 106–111

enabling Spring Data MongoDB 106–107

mapping domain types to documents 107–110

writing interfaces 111

@NotBlank annotation 51

notes() method 418

NotesEndpoint class 418

Notification object 440

NotificationPublisherAware interface 440

O

OAuth 2 187 - 192

oauth2ResourceServer() method 202

objects, creation operations from 287 - 288

OIDC (OpenID Connect) 131

onAfterCreate() method 416, 439

onNext() method 283

-o option 474

order() method 43

@Order annotation 195

OrderController class 47

orderForm() method 45

OrderMessagingService interface 216

OrderProps bean 151

OrderProps class 151

OrderRepository aggregate root service 347-353

OrderRepository interface 73, 111

ordersForUser() controller method 149

OrderSplitter bean 258

org.springframework.data.annotation package 87

origin field 399

origins attribute 166

P

<p> element 38

<packaging> element 456

Page object 312

PageRequest object 149

pageSize property 150

paging 180

<parent> element 14

PasswordEncoder bean 117

PATCH command 171

patchOrder() method 172

path attribute 170

pathMatchers() method 335

@PathVariable annotation 374

@Pattern annotation 52

permitAll() method 126

persisting data

adapting domain for 64–65

mapping domain types for Cassandra 99–105

reactively

document data with MongoDB 353–360

in Cassandra 361–368

R2DBC (Reactive Relational Database Connectivity) 338–353

with Spring Data JPA 85–93

adding to project 85–86

annotating domain as entities 86–89

customizing repositories 90–93

declaring JPA repositories 89–90

placedAt property 149

Płachta, Michał 281

Player object 300

pollRate property 270

post() method 328

@PostAuthorize annotation 136

postForObject() method 184

POSTing resource data 184–185

postprocessing messages 220–222

postProcessMessage() method 232

POST requests 41, 133, 167, 201, 323–324, 399

postTaco() method 170, 319

@PreAuthorize annotation 135

prefix attribute 268

preloading data 70 – 73

private helper method 208

private method 272

processOrder() method 47, 78, 137

Processor interface 283

processPayload() method 273

processRegistration() method 124

processTaco() method 42

prod profile 157

produces attribute 166

ProductService bean 5

@Profile annotation 159

profiles 155–160

activating 158–159

conditionally creating beans with 159–160

profile-specific properties 156–157

project structure 11–17

bootstrapping application 15–16

build specification 12–15

testing application 16–17

property field 399

propertySources field 398

provider property 205

publish() method 455

Publisher interface 282

publishing, reactive types and 294–295

pull model 222

push model 222

put() method 183

PUT command 171

putOrder() method 172

PUTting resources 183–184

Q

qa profile 159

query() method 68

QueueChannel bean 253

R

R2DBC (Reactive Relational Database Connectivity) 338–353

domain entities for 339–343

OrderRepository aggregate root service 347–353

reactive repositories 343–345

testing repositories 345–347

RabbitMQ 226–236

adding to Spring 227–228

receiving message from 232–236

messages with listeners 235–236

with RabbitTemplate 232–235

sending messages with RabbitTemplate 228–232

configuring message converter 231

setting message properties 231–232

*RabbitMQ in Action* (Videla and Williams) 227

*RabbitMQ in Depth* (Roy) 227

RabbitTemplate bean 227

RabbitTemplate.receive() method 232

range() method 290

reactive APIs

consuming REST APIs reactively 325–332

deleting resources 329

exchanging requests 331–332

GETting resources 326–328

handling errors 329–331

sending resources 328–329

functional request handlers 316–320

securing reactive web APIs 333–336

configuring reactive user details service 335–336

configuring reactive web security 333–335

Spring WebFlux 309–316

overview 310–311

writing reactive controllers 312–316

testing reactive controllers 320–325

GET requests 320–323

POST requests 323–324

with live server 324–325

reactively persisting data

document data with MongoDB 353–360

domain document types 354 - 356

reactive MongoDB repositories 356–357

testing reactive MongoDB repositories 357–360

in Cassandra 361–368

creating reactive Cassandra repositories 365–366

domain classes for 362–365

testing reactive Cassandra repositories 366–368

R2DBC 338–353

domain entities for 339–343

OrderRepository aggregate root service 347–353

reactive repositories 343–345

testing repositories 345–347

reactive operations 287–307

combining reactive types 291–295

merging 291–294

selecting first reactive type to publish 294–295

creating reactive types 287–291

from collections 288–290

from objects 287–288

generating Flux data 290–291

logic operations on reactive types 305–307

transforming reactive streams 295–305

buffering data on reactive stream 302–305

filtering data from reactive types 295–299

mapping reactive data 299–302

reactive programming 280–283

Reactive Relational Database Connectivity. *See* R2DBC

Reactive Streams 281–283

Reactor

adding dependencies 286

diagramming reactive flows 285–286

reactive operations 287 – 307

combining reactive types 291–295

creating reactive types 287–291

logic operations on reactive types 305–307

transforming reactive streams 295–305

reactive programming 280–283

receive() method 222

receiveAndConvert() method 223

receiving messages

from JMS 222–226

declaring message listeners 224–226

with JmsTemplate 222–224

from RabbitMQ 232–236

handling RabbitMQ messages with listeners 235–236

with RabbitTemplate 232–235

recents() method 319

recentTacos() method 166, 314

RegisteredClientRepository interface 195

registerForm() method 123

RegistrationController class 122

RegistrationForm object 124

relation names 177

replicas property 450

repositories

customizing 90–93

declaring JPA 89–90

defining JDBC 66–68

interfaces with Spring Data JDBC 79 – 80

reactive

Cassandra 365–368

MongoDB 343–347

Repository interface 79

@RequestBody annotation 170

@RequestMapping annotation 36, 166

request-response servers 375

requests

exchanging 331–332

GET requests 320–323

GETting resources

making requests with base URI 327

timing out on long-running requests 328

HTTP request mappings 400–401

POST requests 323–324

request-stream messaging 376–377

@RequestScope annotation 208

ResourceDatabasePopulator objects 343

resource paths 177

resource property 396

resource servers 201–204

ResponseEntity object 166

REST

consuming 180–185

DELETEing resources 184

GETting resources 182 – 183

POSTing resource data 184–185

PUTting resources 183–184

consuming REST APIs reactively 325–332

deleting resources 329

exchanging requests 331–332

GETting resources 326–328

handling errors 329–331

sending resources 328–329

enabling data-backed services 174–180

adjusting resource paths and relation names 177

paging and sorting 180

securing

creating authorization server 192–201

developing client 204–209

OAuth 2 187–192

securing API with resource server 201–204

writing RESTful controllers 164–174

deleting data from server 173–174

retrieving data from server 164–170

sending data to server 170–171

updating data on server 171–173

restart, automatic application 24

@RestController-annotated classes 175

@RestController annotation 165

@RestResource annotation 179

RestTemplate methods 184

retrieve() method 326

rootProject.name property 445

route() method 317

routerFunction() method 319

routers 256–257

rows, JdbcTemplate 68–70

Roy, Gavin 227

RSocket

creating server and client 372–382

fire-and-forget messages 378–379

request-response 375

request-stream messaging 376–377

sending messages bidirectionally 379–382

overview 370–372

transporting over WebSocket 382–383

run() method 16, 83

RxJava types 314 - 315

S

Sanchez, Illary Huaylupo 426

Saumont, Pierre-Yves 281

save() method 68, 171, 315

saveAll() method 315, 349

saveIngredientRefs() method 76

saveTaco() method 76

schema, with JDBC 70–73

Scott, Dylan 237

security

Actuator 420–421

reactive web APIs 333–336

configuring reactive user details service 335–336

configuring reactive web security 333–335

REST

creating authorization server 192–201

developing client 204–209

OAuth 2 187–192

securing API with resource server 201–204

Spring

applying method-level security 134–136

configuring authentication 116–124

enabling Spring Security 114–116

knowing user 136–139

web requests 125–134

creating custom login page 128–130

enabling third-party authentication 131–133

overview 125–128

preventing cross-site request forgery 133–134

SecurityConfig class 201

SecurityFilterChain bean 125, 204

securityWebFilterChain() method 334

send() method 215

sending messages

bidirectionally 379–382

with KafkaTemplate 238–240

with RabbitTemplate 228–232

configuring message converter 231

setting message properties 231–232

sending resources 328–329

sendNotification() method 440

sendOrder() method 216

Serializable type 110

ServerHttpSecurity object 334

server.port property 145, 399

servers

creating authorization servers 192–201

creating RSocket 372–382

fire-and-forget messages 378–379

request-response 375

request-stream messaging 376–377

sending messages bidirectionally 379–382

deleting data from 173–174

retrieving data from 164–170

securing API with resource server 201–204

sending data to 170–171

testing reactive controllers with live 324–325

updating data on 171–173

server.shutdown property 451

server.ssl.key-store-password property 145

server.ssl.key-store property 145

@ServiceActivator annotation 253

service activators 260–262

@SessionAttributes annotation 38

setAlert() method 379

setNotificationPublisher() method 440

setup() method 352

shouldReturnRecentTacos() method 321

shouldSaveAndFetchOrders() method 360

showDesignForm() method 37

skip() operation 295

@Slf4j annotation 45

Snyder, Bruce 211

sorting 180

source property 221

sources() method 456

 element 39

SpEL (Spring Expression Language) 246

splitters 257–260

Spring

adding RabbitMQ to 227–228

administering

measures and metrics 427–431

securing Admin server 431–433

using Spring Boot Admin 424–427

applications

building and running projects 477

initializing project from command line 472–477

IntelliJ IDEA 463–464

NetBeans 465–469

Spring Tool Suite 459–462

start.spring.io 469–472

applying method-level security 134–136

configuring authentication 116–124

customizing user authentication 119–124

in-memory user details service 118–119

deploying

container images 446–455

executable JAR files 445–446

options for 444–445

WAR files 455–457

enabling Spring Security 114–116

initializing application 6–17

examining project structure 11–17

with Spring Tool Suite 11

knowing user 136–139

landscape 26–28

core Spring Framework 26

Spring Boot 27

Spring Cloud 28

Spring Data 27

Spring Integration and Spring Batch 27–28

Spring Native 28

Spring Security 27

monitoring

creating Actuator MBeans 437–440

sending notifications 440–442

working with Actuator MBeans 435–437

overview 4–6

securing web requests 125–134

creating custom login page 128–130

enabling third-party authentication 131–133

overview 125–128

preventing cross-site request forgery 133–134

setting up for Kafka 237–238

writing application 17–26

building and running 21–23

handling web requests 18

homepages 20

Spring Boot DevTools 23 – 25

testing controller 20–21

spring.activemq.broker-url property 213

spring.activemq.in-memory property 214

Spring application context 4

spring.application.name property 426

spring.jms.template.default-destination property 217

Spring Batch 27–28

Spring Boot 27

Spring Boot Actuator 388–391

configuring base path 389–390

consuming endpoints 391–408

application information 392–395

runtime metrics 405–408

viewing application activity 403–405

viewing configuration details 395–403

customizing 408–419

endpoints 417–419

health indicators 414–415

registering metrics 415–417

the /info endpoint 408–413

enabling and disabling endpoints 390–391

securing 420–421

spring.boot.admin.client.password property 433

spring.boot.admin.client.url property 426

spring.boot.admin.client.username property 433

@SpringBootApplication annotation 15

Spring Boot CLI (command-line interface) 27

@SpringBootConfiguration-annotated class 352

Spring Boot DevTools 23–25

automatic application restart 24

automatic browser refresh and template cache disable 24

built-in H2 console 25

@SpringBootTest annotation 17

Spring Cloud 28

Spring Data 27

Spring Data Cassandra 95–98

spring.data.cassandra.contact-points property 97

spring.data.cassandra.keyspace-name property 97

spring.data.cassandra.local-datacenter property 97

spring.data.cassandra.password property 98

spring.data.cassandra.port property 97

spring.data.cassandra.username property 98

Spring Data JDBC 78–85

adding to build 78–79

annotating domain for persistence 81–83

preloading data with CommandLineRunner 83–85

repository interfaces 79–80

Spring Data JPA 85–93

adding to project 85–86

annotating domain as entities 86–89

customizing repositories 90–93

declaring JPA repositories 89–90

Spring Data MongoDB 106–107

spring.data.rest.base-path property 176

spring.datasource.data property 144

spring.datasource.driver-class-name property 144

spring.datasource.generate-unique-name property 65

spring.datasource.jndi-name property 144

spring.datasource.name property 65

spring.datasource.schema property 144

spring init command 475

Spring Integration 27–28

creating email integration flow 267–274

declaring integration flow 244–251

configuring in Java 247–249

using DSL configuration 249–251

with XML 246–247

landscape 251–267

channel adapters 263–265

endpoint modules 265–267

filters 253–254

gateways 262–263

message channels 252 – 253

routers 256–257

service activators 260–262

splitters 257–260

transformers 254–255

spring.kafka.bootstrap-servers property 238

spring.kafka.template.default-topic property 240

spring.lifecycle.timeout-per-shutdown-phase property 452

spring.main.web-application-type property 274

*Spring Microservices in Action, 2nd Edition* (Carnell and Sanchez) 426

Spring MVC 311

Spring Native 28

spring.profiles.active property 158

spring.profiles property 157

spring.rabbitmq.template.exchange property 230

spring.rabbitmq.template.receive-timeout property 234

spring.rabbitmq.template.routing-key property 230

spring.rsocket.server.port property 374

spring.rsocket.server.transport property 383

Spring Security 27, 114–116

Spring Tool Suite 11, 459–462

Spring WebFlux 309–316

overview 310–311

writing reactive controllers 312–316

handling input reactively 315–316

returning single values 314

RxJava types 314–315

start.spring.io 469–472

StockQuote class 376

streams

request-stream messaging 376–377

transforming reactive streams 295–305

buffering data on reactive stream 302–305

filtering data from reactive types 295–299

mapping reactive data 299–302

String array 300

String parameter 208, 322, 373

String property 141

String value 254

String varargs 84

submissions, form 41-49

subscribe() method 288, 375

Subscriber interface 282

Subscription object 282

systemEnvironment property source 399

T

@Table annotation 101

Taco array 323

tacoById() method 314

Taco class 34, 64, 100, 178, 341

TacoCloudApplication class 15, 56, 456

TacoCloudApplicationTests class 21

tacocloud.ingredients package 402

TacoCounter class 440

Taco domain class 33

Taco entity 177

Taco entity class 340

tacoIds property 348

TacoMetrics bean 415

Taco object 170, 315, 341, 439

TacoOrderAggregateService class 348

TacoOrder class 34, 64, 103, 137, 220, 342

tacoOrderEmailFlow() method 270

TacoOrder entity 177

TacoOrder object 136, 221, 272

taco.orders.pageSize property 150

TacoOrder type 220

Taco parameter 170

TacoRepository.findAllById() method 350

TacoRepository interface 177

tacos.ingredients package 437

tacos package 156

tacos property 348

TacoUDT class 365

tag request attribute 407

take() operation 291, 312

target directory 445

template cache disable, automatic 24

testHomePage() method 21

testing reactive controllers 320–325

GET requests 320–323

POST requests 323–324

with live server 324–325

testTaco() method 321

th:action attribute 134

th:each attribute 39

th:errors attribute 54

th:field attribute 39

th:if attribute 54

th:src attribute 19

third-party authentication 131–133

thread activity 404 – 405

Thymeleaf starter 25

timeout() method 328

toRoman() method 255

toUser() method 124

toUserDetails() method 336

tracing HTTP activity 403–404

transform() method 255

transformation operations 295–305

buffering data on reactive stream 302–305

filtering data from reactive types 295–299

mapping reactive data 299–302

@Transformer annotation 255

transformerFlow() method 255

Transformer interface 255

transformers 254 – 255

@Transient annotation 348

_typeId property 220

U

update() method 68

uppercase() method 263

UpperCaseGateway interface 263

URI, requests with 327

URI parameter 183

uri tag 407

url command-line client 188

User class 119

UserDetails interface 120

UserDetails object 118, 335

userDetailsService() method 122

UserDetailsService bean 118, 194

UserDetailsService interface 118

user-info-uri property 206

UserRepository.findByUsername() method 336

UserRepository object 335

users

configuring authentication 119–124

creating user details service 121–122

defining user domain and persistence 119–121

in-memory user details service 118–119

registering users 122–124

configuring reactive user details service 335–336

security and knowing 136–139

User type 120

V

@Valid annotation 52

validating form input 49–54

declaring validation rules 50–52

displaying validation errors 54

performing validation at form binding 52–54

value attribute 39

<version> element 286

<version> property 447

version property 445

Videla, Alvaro 227

view controllers 54–57

view template library 57–59

Vitale, Thomas 28

W

WAR files, building and deploying 455–457

war plugin 457

web APIs, reactive 333–336

configuring reactive user details service 335–336

configuring reactive web security 333 – 335

web applications

choosing view template library 57–59

displaying information 30–41

creating controller class 34–38

designing view 38–41

establishing domain 31 – 34

processing form submission 41–49

validating form input 49–54

declaring validation rules 50–52

displaying validation errors 54

performing validation at form binding 52–54

working with view controllers 54–57

WebClient bean 327

webEnvironment attribute 325

WebMvcConfigurer interface 56

@WebMvcTest annotation 56

web requests 18, 125–134

creating custom login page 128–130

enabling third-party authentication 131–133

overview 125–128

preventing cross-site request forgery 133–134

WebSocket 382 – 383

websocket() method 383

Web starter 25

WebTestClient() method 323

Williams, Jason J. W. 227

Woolf, Bobby 244

writeToFile() method 245

writing applications 17–26

building and running 21–23

handling web requests 18

homepages 20

Spring Boot DevTools 23 – 25

automatic application restart 24

automatic browser refresh and template cache disable 24

built-in H2 console 25

testing controller 20–21

X

XML 246–247

Z

zip() operation 293

Previous chapter

< Appendix. Bootstrapping Spring applications